```
import os
import sys
import sqlite3
import shutil

from pathlib import Path

from PySide6.QtWidgets import (QMainWindow, QTabWidget, QSplitter, QStatusBar,
                               QLabel, QPushButton, QFontComboBox, QSpinBox,
                               QWidget, QSizePolicy, QApplication, QMessageBox,
                               QFileDialog, QInputDialog, QStyle, QPlainTextEdit, QTextEdit,
                               QAbstractItemView)
from PySide6.QtCore import Qt, QSettings, QDir, Slot, QEvent, QSize, QModelIndex, QThread, QTimer, QCoreApplication
from PySide6.QtGui import (QFont, QIcon, QPalette, QColor, QGuiApplication, QKeySequence,
                           QStandardItemModel, QStandardItem, QTextFormat, QCursor, QFontMetrics, QTextCursor)

# --- Structural Module Imports (Architectural Dependencies) ---
# Ensure these match the actual file naming structure of your project modules
from IndexTreeView import IndexTreeView
from LatexIndexParser import LatexIndexParser
from FileTreeView import FileTreeView
from LatexFilterProxy import LatexFolderFilterProxy
from LatexHighlighter import LatexHighlighter
from LatexIndexWindow import LatexIndexWindow
from ProjectLoadWorker import ProjectLoadWorker
from IndexTreePersistence import IndexTreePersistence
from EditorTab import EditorTab
from IndexTreeController import IndexTreeController
from IndexTextFormatterDelegate import IndexTextFormatterDelegate

INDEX_DATA_DB = 'index_data.db'

class LatexEditor(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("LaTeX Indexing Editor")

        self.settings = QSettings("DH Indexing", "LatexEditor")

        # --- Project Workspace Lifecycle Registers ---
        self.project_root = self.settings.value("last_project_root", "")
        self.project_name = self.settings.value("last_project_name", "")
        self.db_path = ""
        self._tree_modified = False

        # Session tracking structures for hard-drive rolling rollbacks
        self.session_files = set()
        self.backup_registry = {}
        self.backup_dir = None

        # Typographic and Layout Options
        self.current_font_family = self.settings.value("font_family", "Arial")
        self.current_font_size = int(self.settings.value("font_size", 12))
        self.is_dark_mode = self.settings.value("dark_mode", "false") == "true"
        self.current_project_path = self.settings.value("last_project_path", QDir.homePath())

        # Global incremental identification index register offset
        self._current_max_id = 0

        # Initialize Relational Models
        self.base_model = QStandardItemModel(self)
        self.proxy_model = LatexFolderFilterProxy()
        self.proxy_model.setSourceModel(self.base_model)
        self.proxy_model.setDynamicSortFilter(False)

        # Initialize Sidebar Navigation Views
        self.tree_files = FileTreeView(model=self.proxy_model)
        self.tree_index = IndexTreeView() # Holds internal multi-column models
        self.index_controller = IndexTreeController(self.tree_index)
        self.tree_index.setModel(self.index_controller.model)

        self.tree_index.setSelectionMode(QAbstractItemView.SelectionMode.SingleSelection)
        self.tree_index.setSelectionBehavior(QAbstractItemView.SelectionBehavior.SelectItems)
        self.tree_index.setFocusPolicy(Qt.FocusPolicy.StrongFocus)

        # Enable custom context menu triggers on the tree view
        self.tree_index.setContextMenuPolicy(Qt.ContextMenuPolicy.CustomContextMenu)
        self.tree_index.customContextMenuRequested.connect(self.tree_index.show_context_menu)
        self.tree_index.referenceTokenClicked.connect(self.handle_stable_reference_click)

        # Bind QTreeView to our Controller's underlying QAbstractItemModel
        self.tree_index.setModel(self.index_controller.model)
        self.tree_index.setItemDelegateForColumn(0, IndexTextFormatterDelegate(self))

        # Setup Workspace Containers and Layout Splitters
        self.tabs = QTabWidget()
```

```
        self.tabs.setTabsClosable(True)
        self.tabs.setMovable(True)
        self.tabs.setDocumentMode(True)

        self.right_splitter = QSplitter(Qt.Vertical)
        self.right_splitter.addWidget(self.tabs)

        self.latex_index_window = LatexIndexWindow("LaTeX Index", self, self.tabs)
        self.right_splitter.addWidget(self.latex_index_window)
        self.latex_index_window.hide()

        self.splitter = QSplitter(Qt.Horizontal)
        self.splitter.addWidget(self.tree_files)
        self.splitter.addWidget(self.tree_index)
        self.splitter.addWidget(self.right_splitter)
        self.setCentralWidget(self.splitter)

        # Distribute horizontal space constraints (Left sidebars get 15% each, tabs get 70%)
        app_width = self.width()
        tree_width = int(app_width * 0.15)
        right_width = app_width - (tree_width * 2)
        self.splitter.setSizes([tree_width, tree_width, right_width])

        # Distribute vertical space constraints (Tabs get 75%, bottom macro input dock gets 25%)
        app_height = self.height()
        self.right_splitter.setSizes([int(app_height * 0.75), int(app_height * 0.25)])

        # Leave the UI blank and pristine on standard startup:
        self.project_root = ""
        self.project_name = ""
        self.db_path = ""
        self._tree_modified = False
        self._current_max_id = 0

        # STRUCTURAL METADATA REGISTER:
        # Dictionary repository holding format rules used during PDF generation sequences.
        # These default fallbacks initialize the workspace layout if a database is blank.
        self.project_settings = {
            "columns": "2",                # 1, 2, or 3 column layout index parameters
            "heading_format": "\\bfseries", # Font style for letters sections partitions
            "page_style": "fancy",          # Fancyhdr layout style tags
            "compress_ranges": "true",      # Automatically compress sequences (e.g., 4-7 instead of 4,5,6,7)
            "ignore_preamble": "false"      # Skip parsing raw document metadata preamble blocks
        }

        # Safely prepare the default path register so that when the user presses Ctrl+O,
        # it remembers the last directory location they explored.
        self.current_project_path = self.settings.value("last_project_path", QDir.homePath())

        # UI Layout Framing, Custom Menus, and Signal Intercepts
        self.setup_toolbar()
        self.apply_theme()
        self.create_menu()
        self.setup_status_bar()
        self.connect_signals()

        size = QGuiApplication.primaryScreen().availableGeometry().size()
        self.resize(size.width() * 0.75, size.height() * 0.75)

    def setup_status_bar(self):
        self.status_bar = QStatusBar()
        self.setStatusBar(self.status_bar)
        self.file_label = QLabel("Ready.")
        self.status_bar.addWidget(self.file_label)

    def connect_signals(self):
        """Assembles core architectural decoupled signal pathways across components."""
        self.tree_files.doubleClicked.connect(self.open_file)
        self.tree_index.locationRequested.connect(self.go_to_index_location)
        self.tabs.currentChanged.connect(self.update_status_bar)
        self.tabs.tabCloseRequested.connect(self.remove_document_tab)

        # Connect decoupled input widget operations cleanly
        self.latex_index_window.saveRequested.connect(self.handle_widget_save_as_request)
        self.latex_index_window.syncRequested.connect(self.handle_widget_sync_request)

    # --- Project Directory Navigation & Hard-Drive Scanning Operations ---
    def ensure_backup_directory(self, source_file_path: str) -> str:
        """Validates existence of the backup folder structure, recreating it if deleted."""
        # 1. Determine the root directory of the active file or project workspace
        normalized_path = os.path.normpath(source_file_path)
        parent_dir = os.path.dirname(normalized_path)

        # 2. Compute the absolute target for the hidden session backups folder
        backup_root = os.path.join(parent_dir, ".session_backups")

        # 3. Dynamic check and auto-creation safeguard
```

```
    if not os.path.exists(backup_root):
        try:
            os.makedirs(backup_root, exist_ok=True)
            # Apply hidden file attribute on Windows to keep the workspace clean
            if os.name == 'nt':
                import ctypes
                # FILE_ATTRIBUTE_HIDDEN = 0x02
                ctypes.windll.kernel32.SetFileAttributesW(backup_root, 0x02)
        except Exception as e:
            print(f"Critial: Failed to heal missing backup directory folder: {e}")

    return backup_root

def select_folder(self):
    """Prompt user for a root folder, read project names silently if a DB exists, or prompt if new."""
    if self.backup_registry or getattr(self, '_tree_modified', False):
        reply = QMessageBox.question(
            self, 'Uncommitted Changes',
            "The current project has uncommitted modifications. Discard changes?",
            QMessageBox.Yes | QMessageBox.No, QMessageBox.No
        )
        if reply == QMessageBox.No:
            return
        self.revert_session_changes()

    folder = QFileDialog.getExistingDirectory(self, "Select Project Root Folder", self.current_project_path)
    if not folder:
        return

    self.project_root = os.path.normpath(folder)
    self.current_project_path = self.project_root
    self.settings.setValue("last_project_path", self.project_root)

    # Clear active visual layouts to prepare for fresh incoming project payloads
    self.base_model.clear()
    self.tree_index._get_source_model().clear()

    # --- REPAIRED ARCHITECTURAL NAME PERSISTENCE LOOKUP ---
    # Scan the folder to see if any project name database file footprints already exist
    existing_db_file = None
    for filename in os.listdir(self.project_root):
        if filename.endswith(INDEX_DATA_DB):
            existing_db_file = os.path.join(self.project_root, filename)
            break

    if existing_db_file and os.path.exists(existing_db_file):
        # A project database exists. Silently parse the metadata table to extract the title name!
        # Silently read the workspace name and all formatting parameters from the file
        proj_name_str, loaded_settings = IndexTreePersistence.read_metadata(existing_db_file)

        if proj_name_str:
            self.project_name = proj_name_str
            self.db_path = existing_db_file

            # Update your main window format registry with the values loaded from disk
            self.project_settings.update(loaded_settings)
            print(f"DEBUG: Silently loaded project settings profile for: {self.project_name}")
            print(f"DEBUG: Active Formatting Settings Matrix: {self.project_settings}")
        else:
            self.project_name = os.path.basename(self.project_root)
            self.db_path = existing_db_file
    else:
        # SCENARIO: Brand new workspace folder. Prompt the user for an initial configuration title.
        default_name = os.path.basename(self.project_root)
        proj_name, ok = QInputDialog.getText(
            self, "New Project Configuration", "Enter unique name for this new project:", text=default_name
        )
        if not ok or not proj_name.strip():
            print("DEBUG: Project creation sequence aborted due to empty name configuration.")
            return

        self.project_name = proj_name.strip()
        self.db_path = os.path.join(self.project_root, f"{self.project_name}_index_data.db")

    # 4. Configure unified workspace runtime structures
    self.backup_dir = os.path.join(self.project_root, ".session_backups")
    os.makedirs(self.backup_dir, exist_ok=True)

    # Re-build left-hand project tree browser listing vectors
    self._scan_folder(self.project_root, self.base_model.invisibleRootItem())
    self.tree_files.show()
    self.tree_files.expandAll()

    # Gather file targets
    discovered_tex_files = []
    for root_dir, _, filenames in os.walk(self.project_root):
        if ".session_backups" in root_dir:
```

```
            continue
        for name in filenames:
            if name.lower().endswith('.tex'):
                discovered_tex_files.append(os.path.join(root_dir, name))

    # 5. Hydrate everything smoothly using your background threading worker engine
    self.open_project_workspace(self.db_path, discovered_tex_files)

def _scan_folder(self, path, parent_item):
    """Recursively gathers folder assets onto tree standard models while avoiding backup noise."""
    try:
        for entry in os.scandir(path):
            if entry.name == ".session_backups":
                continue

            item = QStandardItem(entry.name)
            item.setData(entry.is_dir(), Qt.UserRole)
            item.setData(entry.path, Qt.UserRole + 1)

            icon = QStyle.SP_DirIcon if entry.is_dir() else QStyle.SP_FileIcon
            item.setIcon(self.style().standardIcon(icon))

            parent_item.appendRow(item)
            if entry.is_dir():
                self._scan_folder(entry.path, item)
    except PermissionError:
        pass

# --- Index Reconstitution, DB Tracking, & Multi-Column Persistence Routing ---

def load_project_indexing_state(self, file_paths: list[str]):
    """Sweeps source structures using the fallback regex core and generates sequential records."""
    self.tree_index.setSortingEnabled(False)
    model = self.tree_index._get_source_model()
    model.clear()
    model.setHorizontalHeaderLabels(["Index Terms", "References..."])

    self._current_max_id = 0
    running_id_pool = 1

    for path in file_paths:
        normalized_target = os.path.normpath(path)
        payloads, next_id = LatexIndexParser.parse_file(normalized_target, start_id=running_id_pool)
        running_id_pool = next_id

        for parts_list, uid_dict in payloads:
            self.tree_index.add_index_to_tree(parts_list, uid_dict)

    self._current_max_id = running_id_pool - 1

    self.tree_index.setSortingEnabled(True)
    model.sort(0, Qt.AscendingOrder)
    self.tree_index.expandAll()

# Replace open_project_workspace and load_project_indexing_state in LatexEditor with this:
def open_project_workspace(self, project_db_path: str, tex_file_paths: list[str]):
    """Launches a non-blocking background thread worker loop to handle folder ingestion paths."""
    self.db_path = os.path.normpath(project_db_path)
    self.tree_index.project_root = self.project_root

    # Provide clear interface feedback and block interactions during initialization passes
    self.status_bar.showMessage("Initializing background loader context...", 0)
    self.centralWidget().setEnabled(False) # Prevents clicks while loading occurs

    # Instantiate the secondary QThread execution context container
    self._load_thread = QThread()
    self._load_worker = ProjectLoadWorker(self.db_path, self.project_root, tex_file_paths)

    # Move our worker object down onto the thread context pipeline register
    self._load_worker.moveToThread(self._load_thread)

    # Establish thread communication connection linkages
    self._load_thread.started.connect(self._load_worker.process)
    self._load_worker.statusUpdated.connect(self.handle_worker_status_update)
    self._load_worker.errorOccurred.connect(self.handle_worker_error)
    self._load_worker.finished.connect(self.handle_worker_finished)
    # self._load_worker.finished.connect(self.on_background_indexing_finished)

    # Enforce resource reclamation cleanups when processes exit the pipeline registers
    self._load_worker.finished.connect(self._load_thread.quit)
    self._load_worker.finished.connect(self._load_worker.deleteLater)
    self._load_thread.finished.connect(self._load_thread.deleteLater)

    # Send execution sequences into background
    self._load_thread.start()

@Slot(object, int)
```

```
def handle_stable_reference_click(self, proxy_index, click_x: int):
    """
    Processes clicked metrics, synchronizes the correct active editor tab,
    and dispatches pristine coordinate jumps across multiple files.
    """
    # 1. Resolve proxy model index to source model index mapping
    if hasattr(self, 'index_proxy_model'):
        source_index = self.index_proxy_model.mapToSource(proxy_index)
    else:
        source_index = proxy_index

    ROLE_UID_DATA = Qt.ItemDataRole.UserRole + 1
    records = source_index.data(ROLE_UID_DATA)
    display_text = source_index.data(Qt.ItemDataRole.DisplayRole) or ""

    if not records or not display_text:
        return

    # 2. Tokenize the visual text row using font metrics to identify the targeted reference
    font = self.tree_index.font()
    metrics = QFontMetrics(font)
    tokens = display_text.split(" ")
    current_offset_x = 0
    matched_record = None

    for token in tokens:
        if not token:
            continue
        token_width = metrics.horizontalAdvance(token)
        space_width = metrics.horizontalAdvance(" ")

        if current_offset_x <= click_x <= (current_offset_x + token_width):
            try:
                # Parse the numerical ID hidden in bracket segments like [495]
                target_uid = int(token.strip("[]"))
                for rec in records:
                    # Checks both potential schema variants generated by the parser
                    rec_id = rec.get("id") or rec.get("unique_id_number")
                    if rec and rec_id is not None and int(rec_id) == target_uid:
                        matched_record = rec
                        break
            except ValueError:
                pass
            break
        current_offset_x += token_width + space_width

    # 3. Securely Route the Viewport Focus Across Files
    if matched_record:
        target_file = self.clean_windows_path(matched_record.get("file_path"))
        line_num = matched_record.get("line_number")
        col_offset = matched_record.get("column_offset")

        # Scan tab layout to determine if the targeted file sheet is already active
        tab_found = False
        for idx in range(self.tabs.count()):
            editor_tab = self.tabs.widget(idx)
            if editor_tab and hasattr(editor_tab, 'file_path') and editor_tab.file_path == target_file:
                self.tabs.setCurrentIndex(idx) # Switch context focus immediately
                tab_found = True
                break

        # Lazy load the text tab container if it isn't already visible
        if not tab_found:
            file_name = Path(target_file).name
            self.create_editor_tab(target_file, file_name)

        # Let the PySide6 rendering window finalize its tab swapping geometries
        QCoreApplication.processEvents()

        # 4. Dispatch the Precise Text Target Jump Instruction
        target_editor = self.tabs.currentWidget()
        print(f"DEBUG: matched_record: {matched_record}")

        if target_editor and hasattr(target_editor, "jump_to_coordinates"):
            target_editor.jump_to_coordinates(
                line = line_num,
                column = col_offset,
                absolute_position = matched_record.get("absolute_position"), # Feed absolute index!
                is_one_indexed = True
            )

@Slot(str)
def handle_worker_status_update(self, msg: str):
    """Updates your existing text label status bar elements live with clear feedback."""
    self.file_label.setText(msg)

@Slot(str)
```

```
def handle_worker_error(self, err_msg: str):
    """Cleans window interaction lockdowns if background execution fails."""
    self.centralWidget().setEnabled(True)
    self.file_label.setText("Ready.")
    QMessageBox.critical(self, "Project Loading Failure", f"An out-of-thread error occurred:\n{err_msg}")

@Slot(bool, list, list, list)
def handle_worker_finished(self, is_db_restored: bool, headings: list, references: list, file_tree_data: list):
    """Receives completed thread data arrays and hydrates the UI views using a unified controller layout."""
    # Re-sync main window tracking variables back to verified thread strings
    if hasattr(self._load_worker, 'db_path'):
        self.db_path = str(self._load_worker.db_path)

        import os
        filename = os.path.basename(self.db_path)
        if "_index data.db" in filename:
            self.project_name = filename.replace("_index data.db", "")

    # Populate left-hand File Browser View model
    self.base_model.clear()
    self._rebuild_file_tree_view(file_tree_data, self.base_model.invisibleRootItem())
    self.tree_files.show()
    self.tree_files.expandAll()

    # Route data arrays directly to the unified controller
    if hasattr(self, 'index_controller'):
        self.index_controller.populate_from_worker_payloads(headings, references)
    else:
        print("ERROR: index_controller is not instantiated on MainWindow context.")

    # If data was derived dynamically from regex fallback sweeps, cache it back to persistence
    if not is_db_restored:
        # Force auto-save compilation transaction blocks back onto the database file surface
        self.tree_index.save_to_db(self.db_path)
    else:
        self.clear_session_backups()

    self._tree_modified = False

    # 4. Synchronize counters and release window interaction locks
    self.initialize_id_counter()

    # Link project coordinates down to your IndexTreeView wrapper class
    self.tree_index.project_root = self.project_root
    self.tree_index.expandAll()

    self.centralWidget().setEnabled(True)
    self.file_label.setText("Ready.")
    self.status_bar.showMessage("Project loaded successfully.", 3000)

def _rebuild_file_tree_view(self, flat_nodes_list: list, parent_qt_item):
    """Assembles QStandardItems safely on the main thread from raw background worker list scopes."""
    from PySide6.QtGui import QStandardItem
    from PySide6.QtWidgets import QStyle

    for node in flat_nodes_list:
        item = QStandardItem(node["name"])
        item.setData(node["is_dir"], Qt.UserRole)
        item.setData(node["path"], Qt.UserRole + 1)

        icon = QStyle.SP_DirIcon if node["is_dir"] else QStyle.SP_FileIcon
        item.setIcon(self.style().standardIcon(icon))

        parent_qt_item.appendRow(item)
        if node["is_dir"] and node["children"]:
            self._rebuild_file_tree_view(node["children"], item)

def initialize_id_counter(self):
    """Finds the absolute highest reference integer token down within tables."""
    if not os.path.exists(self.db_path):
        self._current_max_id = 0
        return

    try:
        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()
        cursor.execute("SELECT MAX(unique_id_number) FROM index_references")
        result = cursor.fetchone()
        conn.close()

        if result and result[0] is not None:
            self._current_max_id = int(result[0])
        else:
            self._current_max_id = 0
    except sqlite3.OperationalError:
        self._current_max_id = 0
```

```
    def get_and_increment_id(self) -> int:
        """Increments the local project sequence counter and returns the token value cleanly."""
        self._current_max_id += 1
        print(f"DEBUG: Dispatched next unique sequential project ID token: {self._current_max_id}")
        return self._current_max_id

    # --- Document Tab Manipulation & Frame Focus Control ---
    def clean_windows_path(self, raw_path: str) -> str:
        """Removes hidden control characters, stripping enclosing quotes and anomalies."""
        if not raw_path:
            return ""
        # Cast to string and strip enclosing quote artifacts or white-spaces
        cleaned = str(raw_path).strip().strip("'\"")

        # Eliminate structural control character leakage (null bytes, newlines)
        cleaned = cleaned.replace('\x00', '').replace('\n', '').replace('\r', '')

        # Convert forward slashes to system native backslashes safely
        return os.path.normpath(str(Path(cleaned).resolve()))

    @Slot(QModelIndex)
    def open_file(self, proxy_index):
        """Maps view clicks back to disk paths and opens document canvas sheets."""
        if not proxy_index.isValid():
            return

        # Determine if the click originated from the Index Tree View vs File Tree View
        is_index_click = (proxy_index.model() == self.tree_index.model() or
                          hasattr(self, 'tree_index') and self.tree_index.model() and
                          proxy_index.model() == self.tree_index.model().proxyModel() if hasattr(self.tree_index.model(),
'proxyModel') else False)

        # Standard source index mapping through QSortFilterProxyModel layers
        sender = self.sender()
        if sender == self.tree_index or proxy_index.column() == 1:
            # Handle column 1 embedded token click execution routing
            self._handle_index_reference_double_click(proxy_index)
            return

        # Fallback to your original file-tree double-click behavior for folder items
        source_index = self.proxy_model.mapToSource(proxy_index)
        item = self.base_model.itemFromIndex(source_index)
        if not item:
            return

        is_dir = item.data(Qt.UserRole)
        path = item.data(Qt.UserRole + 1)

        if not is_dir and path:
            normalized_path = os.path.normpath(path)
            for i in range(self.tabs.count()):
                if os.path.normpath(self.tabs.tabToolTip(i)) == normalized_path:
                    self.tabs.setCurrentIndex(i)
                    return
            self.create_editor_tab(normalized_path, item.text())

    def create_editor_tab(self, path, name):
        """Spawns an independent file sheet view with custom typeface properties."""
        try:
            # Re-verify layout validation
            normalized_path = self.clean_windows_path(path)

            with open(normalized_path, 'r', encoding='utf-8') as f:
                editor = EditorTab()
                # Enable smart word wrapping based on the visible widget boundaries
                editor.setLineWrapMode(QPlainTextEdit.LineWrapMode.WidgetWidth)
                editor.setFont(QFont(self.current_font_family, self.current_font_size))
                editor.setPlainText(f.read())

                editor.file_path = normalized_path
                editor.viewport().installEventFilter(self)
                editor.highlighter = LatexHighlighter(editor.document(), self.is_dark_mode)
                editor.cursorPositionChanged.connect(self.update_status_bar)

                idx = self.tabs.addTab(editor, name)
                self.tabs.setTabToolTip(idx, normalized_path)
                self.tabs.setCurrentIndex(idx)

                self.update_menu_item_state()
        except Exception as e:
            self.status_bar.showMessage(f"Error opening file: {e}", 5000)
            print(f"Detailed Debug: open() crashed on string: {repr(path)} -> Cleaned: {repr(normalized_path)}")

    def _handle_index_reference_double_click(self, proxy_index):
        """Computes reference boundaries dynamically using font metrics on double-click."""
        # Unpack proxy model back to raw index tree source structure if using one
        if hasattr(self, 'index_proxy_model'):
```

```
        source_index = self.index_proxy_model.mapToSource(proxy_index)
    else:
        source_index = proxy_index

    if source_index.column() != 1:
        return  # Ignore heading terms double-clicks in column 0

    # Retrieve the collection of structured reference dictionaries stored via UserRole
    records = source_index.data(Qt.UserRole + 1)
    display_text = source_index.data(Qt.DisplayRole) or ""
    if not records or not display_text:
        return

    # Fetch viewport coordinates to compute exactly where the mouse pointer fell
    visual_rect = self.tree_index.visualRect(proxy_index)
    cursor_pos = self.tree_index.viewport().mapFromGlobal(QCursor.pos())
    click_x = cursor_pos.x() - visual_rect.x()

    # Measure token dimensions using index row typography properties
    font = self.tree_index.font()
    metrics = QFontMetrics(font)

    tokens = display_text.split(" ")
    current_offset_x = 0
    matched_record = None

    for token in tokens:
        if not token:
            continue

        token_width = metrics.horizontalAdvance(token)
        space_width = metrics.horizontalAdvance(" ")

        # Check if mouse click coordinates reside inside the specific token's text margins
        if current_offset_x <= click_x <= (current_offset_x + token_width):
            try:
                # Isolate clean integer ID values by removing surrounding brackets
                target_uid = int(token.strip("[]"))
                for rec in records:
                    if rec and int(rec.get("unique_id_number", -1)) == target_uid:
                        matched_record = rec
                        break
            except ValueError:
                pass
            break

        current_offset_x += token_width + space_width

    # Dispatch view manager commands if geometry constraints align to an embedded item
    if matched_record:
        self.navigate_to_embedded_index_coordinate(
            file_path=matched_record.get("file_path"),
            line_num=matched_record.get("line_number"),
            col_offset=matched_record.get("column_offset")
        )

def navigate_to_embedded_index_coordinate(self, file_path: str, line_num: int, col_offset: int):
    """Locates tab indexes, opens documents, and schedules a deferred jump execution."""
    if not file_path or line_num is None:
        return
    print(f"DEBUG (navigate)- file: {file_path}, line: {line_num}, col: {col_offset}")
    normalized_target = os.path.normpath(file_path)
    active_editor = None

    # 1. Search for an open workspace sheet matching target paths
    for i in range(self.tabs.count()):
        if os.path.normpath(self.tabs.tabToolTip(i)) == normalized_target:
            self.tabs.setCurrentIndex(i)
            active_editor = self.tabs.widget(i)
            break

    # 2. Open document tab sheets if hidden inside background paths
    if not active_editor:
        tab_name = Path(normalized_target).name
        self.create_editor_tab(normalized_target, tab_name)
        active_editor = self.tabs.currentWidget()

    # 3. CRITICAL DEFERRAL FIX: Do not manipulate the cursor immediately.
    # Defer execution to let tab sheets, finding dialog overlays, and panel splitters
    # completely finish resizing layout geometries before the cursor math runs.

    # Use lambda execution blocks to forward coordinate profiles cleanly downstream
    QTimer.singleShot(40, lambda: self._execute_deferred_text_jump(active_editor, line_num, col_offset))

def _execute_deferred_text_jump(self, active_editor, line_num: int, col_offset: int):
    """Executes cursor positioning, highlighting, and viewport centering on a settled canvas layout."""
    if not active_editor or not hasattr(active_editor, "document"):
```

```
            return

        # Correct for 1-indexed document coordinates mapping to 0-indexed text fields safely
        target_line = max(0, int(line_num) - 1)
        target_col = max(0, int(col_offset) - 1)

        # Temporarily drop signals to stop secondary status tracking updates from triggering mid-layout
        active_editor.blockSignals(True)

        try:
            document = active_editor.document()

            # Reset selection properties to baseline states
            blank_cursor = active_editor.textCursor()
            blank_cursor.clearSelection()
            active_editor.setTextCursor(blank_cursor)

            block = document.findBlockByLineNumber(target_line)
            if block.isValid():
                # Compute absolute location within the character array buffer
                absolute_start_pos = block.position() + min(target_col, block.length() - 1)

                # Fetch raw text lines content to evaluate macro brace tracking bounds
                line_text = block.text()
                remaining_text = line_text[target_col:]

                selection_length = len(r"\index")

                # Trace complete macro parameters closure to frame highlighting parameters
                if remaining_text.startswith(r"\index"):
                    brace_count = 0
                    has_opened = False
                    for idx, char in enumerate(remaining_text):
                        if char == '{':
                            brace_count += 1
                            has_opened = True
                        elif char == '}':
                            brace_count -= 1

                        if has_opened and brace_count == 0:
                            selection_length = idx + 1
                            break

                # Relocate view cursor positions safely and apply highlighting spans
                cursor = active_editor.textCursor()
                cursor.setPosition(absolute_start_pos)
                cursor.setPosition(absolute_start_pos + selection_length, QTextCursor.MoveMode.KeepAnchor)

                # Apply selection layout back to view canvas configurations
                active_editor.setTextCursor(cursor)

                # Reset the page horizontal bars to zero to cancel out text wrap offsets
                active_editor.horizontalScrollBar().setValue(0)

                # Force view scroll containers to center explicitly over target tags selection bounds
                active_editor.centerCursor()

        finally:
            active_editor.blockSignals(False)

        # Grant focus to the tab text view after layout transformations settle
        active_editor.setFocus(Qt.FocusReason.OtherFocusReason)

    @Slot(str, int, int)
    def go_to_index_location(self, file_path: str, line_number: int, column_number: int):
        """Processes double-click events by scrubbing text parameters and routing views safely."""
        # 1. Neutralize structural path errors immediately
        normalized_target = self.clean_windows_path(file_path)

        if not normalized_target or normalized_target == ".":
            self.status_bar.showMessage("Cannot navigate: Extracted index path string is invalid.", 4000)
            return

        target_editor = None

        # 2. Inspect active tab components using scrubbed parameters
        for i in range(self.tabs.count()):
            tab_path = self.clean_windows_path(self.tabs.tabToolTip(i))
            if tab_path == normalized_target:
                self.tabs.setCurrentIndex(i)
                target_editor = self.tabs.widget(i)
                break

        # 3. Fallback: Create a new tab if the target file isn't open
        if not target_editor:
            target_path_obj = Path(normalized_target)
            file_name = target_path_obj.name
```

```
        # Use raw OS check to bypass caching issues
        if not os.path.isfile(normalized_target):
            self.status_bar.showMessage(f"Path not found: '{file_name}' at {normalized_target}", 5000)
            print(f"Debug Fail: System cannot resolve physical path validation: {repr(normalized_target)}")
            return

        self.create_editor_tab(normalized_target, file_name)
        target_editor = self.tabs.currentWidget()

    if target_editor:
        self.move_and_highlight(target_editor, line_number, column_number)
        self.save_tex_file_to_disk(target_editor, normalized_target)

def move_and_highlight(self, editor, line, col):
    """Positions text cursors cleanly, highlights target lines, and centers active targets."""
    target_line = max(1, int(line))
    target_col = max(1, int(col))

    # 1. Position and focus the active text cursor
    block = editor.document().findBlockByLineNumber(target_line - 1)
    cursor = editor.textCursor()
    cursor.setPosition(block.position() + (target_col - 1))

    editor.setTextCursor(cursor)
    editor.setFocus()
    editor.centerCursor()
    # editor.ensureCursorVisible()

    # 2. Configure line-highlight visual layer (ExtraSelection)
    selection = QTextEdit.ExtraSelection()

    # Set background fill color to a soft translucent yellow for readability
    # This colour works well for both light and dark mode
    highlight_color = QColor(255, 255, 0, 100)
    selection.format.setBackground(highlight_color)
    selection.format.setProperty(QTextFormat.Property.FullWidthSelection, True)

    # Map the selection bounds precisely to the targeted line block
    selection.cursor = editor.textCursor()
    selection.cursor.clearSelection()

    # Apply the layer to the editor viewport
    editor.setExtraSelections([selection])

def remove_document_tab(self, index):
    self.tabs.removeTab(index)
    self.update_menu_item_state()

def handle_add_subheading_workflow(self, item):
    print(f"MAIN WINDOW WORKFLOW: Launching additions dialog prompt under branch path: {item.text()}")

def handle_delete_workflow(self, item):
    print(f"MAIN WINDOW WORKFLOW: Running deletion sweeps on source docs for entry term: {item.text()}")

def toggle_file_sidebar(self):
    self.tree_files.setVisible(not self.tree_files.isVisible())

def toggle_index_sidebar(self):
    self.tree_index.setVisible(not self.tree_index.isVisible())

# --- Layout Theming & Typography Modifications ---

def toggle_dark_mode(self):
    self.is_dark_mode = not self.is_dark_mode
    self.apply_theme()
    self.settings.setValue("dark_mode", str(self.is_dark_mode).lower())

def apply_theme(self):
    app = QApplication.instance()
    if self.is_dark_mode:
        app.setStyle("Fusion")
        palette = QPalette()
        palette.setColor(QPalette.Window, QColor(53, 53, 53))
        palette.setColor(QPalette.WindowText, Qt.white)
        palette.setColor(QPalette.Base, QColor(25, 25, 25))
        palette.setColor(QPalette.AlternateBase, QColor(53, 53, 53))
        palette.setColor(QPalette.ToolTipBase, Qt.white)
        palette.setColor(QPalette.ToolTipText, Qt.white)
        palette.setColor(QPalette.Text, Qt.white)
        palette.setColor(QPalette.Button, QColor(53, 53, 53))
        palette.setColor(QPalette.ButtonText, Qt.white)
        palette.setColor(QPalette.BrightText, Qt.red)
        palette.setColor(QPalette.Link, QColor(42, 130, 218))
        palette.setColor(QPalette.Highlight, QColor(42, 130, 218))
        palette.setColor(QPalette.HighlightedText, Qt.black)
        app.setPalette(palette)
```

```
            self.setStyleSheet("QMainWindow { background-color: #353535; }")
            self.tabs.setStyleSheet("QTabWidget::pane { border: 1px solid #444; background: #252525; }")

            tree_style = "QTreeView { background-color: #191919; color: white; } QHeaderView::section { background-color: #353535;
color: white; border: 1px solid #444; }"
            self.tree_files.setStyleSheet(tree_style)
            self.tree_index.setStyleSheet(tree_style)
        else:
            app.setPalette(app.style().standardPalette())
            self.setStyleSheet("")
            self.tabs.setStyleSheet("")
            self.tree_files.setStyleSheet("")
            self.tree_index.setStyleSheet("")

        for i in range(self.tabs.count()):
            editor = self.tabs.widget(i)
            if hasattr(editor, 'highlighter'):
                editor.highlighter.set_dark_mode(self.is_dark_mode)

    def setup_toolbar(self):
        icon_size = 32
        self.toolbar = self.addToolBar("Main")

        self.dark_toggle = QPushButton()
        self.dark_toggle.setIcon(QIcon("icons/night-mode.png"))
        self.dark_toggle.setIconSize(QSize(icon_size, icon_size))
        self.dark_toggle.setShortcut("Ctrl+Shift+D")
        self.dark_toggle.setToolTip("Toggle Dark Mode (Ctrl+Shift+D)")
        self.dark_toggle.setCheckable(True)
        self.dark_toggle.setChecked(self.is_dark_mode)
        self.dark_toggle.clicked.connect(self.toggle_dark_mode)
        self.toolbar.addWidget(self.dark_toggle)

        self.toolbar.addSeparator()

        self.file_toggle = QPushButton()
        self.file_toggle.setIcon(QIcon("icons/folder-open.png"))
        self.file_toggle.setIconSize(QSize(icon_size, icon_size))
        self.file_toggle.setToolTip("Toggle File Sidebar (Ctrl+B)")
        self.file_toggle.setCheckable(True)
        self.file_toggle.setChecked(True)
        self.file_toggle.clicked.connect(self.toggle_file_sidebar)
        self.toolbar.addWidget(self.file_toggle)

        self.toolbar.addSeparator()

        self.index_toggle = QPushButton()
        self.index_toggle.setIcon(QIcon("icons/tree-view.svg"))
        self.index_toggle.setIconSize(QSize(icon_size, icon_size))
        self.index_toggle.setToolTip("Toggle Index Sidebar (Ctrl+Shift+I)")
        self.index_toggle.setCheckable(True)
        self.index_toggle.setChecked(True)
        self.index_toggle.clicked.connect(self.toggle_index_sidebar)
        self.toolbar.addWidget(self.index_toggle)

        self.toolbar.addSeparator()

        self.font_picker = QFontComboBox()
        self.font_picker.setCurrentFont(QFont(self.current_font_family))
        self.font_picker.currentFontChanged.connect(self.update_font_family)
        self.toolbar.addWidget(QLabel(" Font: "))
        self.toolbar.addWidget(self.font_picker)

        self.size_picker = QSpinBox()
        self.size_picker.setRange(8, 72)
        self.size_picker.setValue(self.current_font_size)
        self.size_picker.valueChanged.connect(self.update_font_size)
        self.toolbar.addWidget(QLabel(" Size: "))
        self.toolbar.addWidget(self.size_picker)

        spacer = QWidget()
        spacer.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Preferred)
        self.toolbar.addWidget(spacer)

        self.close_btn = QPushButton()
        self.close_btn.setIcon(QIcon("icons/exit-app.svg"))
        self.close_btn.setIconSize(QSize(icon_size, icon_size))
        self.close_btn.setToolTip("Close Application (Alt+F4)")
        self.close_btn.clicked.connect(self.close) # Reroutes correctly through closeEvent
        self.toolbar.addWidget(self.close_btn)

    def update_font_family(self, font):
        self.current_font_family = font.family()
        self.settings.setValue("font_family", self.current_font_family)
        self.apply_font_to_all()
```

```
    def update_font_size(self, size):
        self.current_font_size = size
        self.size_picker.blockSignals(True)
        self.size_picker.setValue(size)
        self.size_picker.blockSignals(False)
        self.status_bar.showMessage(f"Font size: {size}pt", 2000)
        self.settings.setValue("font_size", self.current_font_size)
        self.apply_font_to_all()

    def apply_font_to_all(self):
        new_font = QFont(self.current_font_family, self.current_font_size)
        for i in range(self.tabs.count()):
            editor = self.tabs.widget(i)
            if isinstance(editor, EditorTab):
                editor.setFont(new_font)
            # if isinstance(editor, QPlainTextEdit):
            #     editor.setFont(new_font)

    def create_menu(self):
        self.menuBar().setStyleSheet("QMenu::item:disabled { color: #888888; background-color: transparent; }")
        file_menu = self.menuBar().addMenu("&File")
        file_menu.addAction("&Open Folder", self.select_folder, "Ctrl+O")
        file_menu.addSeparator()
        file_menu.addAction("&Exit", self.close, "Alt+F4")

        edit_menu = self.menuBar().addMenu("&Edit")
        edit_menu.addAction("&Find...", self.route_find_to_active_tab, QKeySequence.StandardKey.Find)

        view_menu = self.menuBar().addMenu("&View")
        view_menu.addAction("Toggle &File Sidebar", self.toggle_file_sidebar, "Ctrl+B")
        view_menu.addAction("Toggle &Index Sidebar", self.toggle_index_sidebar, "Ctrl+Shift+I")
        self.index_entry_action = view_menu.addAction("Toggle Index &Entry Window", self.toggle_index_entry_window,
QKeySequence(Qt.CTRL | Qt.Key_Backslash))
        self.update_menu_item_state()

    def update_menu_item_state(self):
        self.index_entry_action.setEnabled(self.tabs.count() > 0)

    @Slot()
    def update_status_bar(self):
        """
        Updates the application status bar metrics.
        Computes explicit physical line lines rather than internal paragraph layouts.
        """
        editor = self.tabs.currentWidget()
        if not editor or not isinstance(editor, EditorTab):
            return

        cursor = editor.textCursor()

        # 1. Grab the current active text block context
        block = cursor.block()
        logical_block_num = block.blockNumber() # 0-indexed logical block

        # 2. Extract the layout framework engine inside this block
        layout = block.layout()
        relative_pos_in_block = cursor.position() - block.position()

        # Determine exactly which visual sub-line line-wrap row the cursor sits on
        visual_line_in_block = layout.lineForTextPosition(relative_pos_in_block).lineNumber()

        # 3. Compute the cumulative true row count up to this point
        true_physical_line = 1
        for block_idx in range(logical_block_num):
            prev_block = editor.document().findBlockByLineNumber(block_idx)
            if prev_block.isValid() and prev_block.layout():
                # Add the number of wrapped lines inside every preceding block
                true_physical_line += prev_block.layout().lineCount()
            else:
                true_physical_line += 1

        # Append the line wrap sub-index within the active block
        true_physical_line += visual_line_in_block

        # 4. Extract Column positioning
        current_column = cursor.columnNumber() + 1

        # Commit clean metric display to user status pipeline
        self.status_bar.showMessage(f"Line: {true_physical_line} | Col: {current_column}")

    def toggle_index_entry_window(self):
        if self.tabs.count() > 0:
            self.latex_index_window.hide() if self.latex_index_window.isVisible() else self.latex_index_window.show()
        else:
            self.latex_index_window.hide()

    # --- Decoupled Input Widget Signal Intercept Handlers ---
```

```
@Slot(object, list)
def handle_widget_save_as_request(self, editor, result_holder: list):
    """Intercepts an 'Untitled' save request, updates metadata, and returns status flags."""
    success = self.save_file_as(editor)
    result_holder.append(success)

@Slot(object, str)
def handle_widget_sync_request(self, editor, file_path: str):
    """Silently commits active insertions directly onto project files."""
    self.save_tex_file_to_disk(editor, file_path)
    self._tree_modified = True

def save_file_as(self, editor) -> bool:
    """Standard file save dialog fallback wrapper to assign paths to raw buffers."""
    file_path, _ = QFileDialog.getSaveFileName(self, "Save LaTeX Source File", self.current_project_path, "LaTeX Files (*.tex)")
    if not file_path:
        return False

    norm_path = os.path.normpath(file_path)
    editor.file_path = norm_path

    current_index = self.tabs.indexOf(editor)
    if current_index != -1:
        self.tabs.setTabToolTip(current_index, norm_path)
        self.tabs.setTabText(current_index, os.path.basename(norm_path))

    # Refresh the left file explorer list to reflect the new asset addition
    self.base_model.clear()
    self._scan_folder(self.project_root, self.base_model.invisibleRootItem())
    self.tree_files.expandAll()

    return self.save_tex_file_to_disk(editor, norm_path)

# --- Session State Isolated Control Engine (Pristine Rollbacks System) ---

def register_file_for_session(self, file_path):
    """Creates an un-mutated file copy within hidden backup structures at session start."""
    norm_path = os.path.normpath(file_path)
    if norm_path in self.session_files:
        return

    self.session_files.add(norm_path)

    if os.path.exists(norm_path) and self.backup_dir:
        backup_filename = f"backup_{len(self.backup_registry)}_{os.path.basename(norm_path)}"
        backup_dest = os.path.join(self.backup_dir, backup_filename)
        shutil.copy2(norm_path, backup_dest)
        self.backup_registry[norm_path] = backup_dest

def save_tex_file_to_disk(self, editor, file_path) -> bool:
    """Commits active memory bytes to local files so background tools compile smoothly."""
    self.register_file_for_session(file_path)
    try:
        # 1. Commit active memory layout out to the primary document file
        with open(file_path, 'w', encoding='utf-8') as f:
            f.write(editor.toPlainText())

        # 2. Intercept and heal missing backup directory infrastructure on the fly
        try:
            import shutil
            backup_root = self.ensure_backup_directory(file_path)
            backup_dest = os.path.join(backup_root, os.path.basename(file_path))

            # Copy updated document out to the synchronized session engine
            shutil.copy2(file_path, backup_dest)
        except Exception as backup_err:
            # Use status bar for background tasks to avoid breaking the editing flow
            self.status_bar.showMessage(f"Session backup skipped: {backup_err}", 4000)
            print(f"Debug: Minor session cache sync error -> {backup_err}")

        return True

    except Exception as e:
        QMessageBox.critical(self, "Save Error", f"Could not save text file: {e}")
        return False

def commit_session_to_database(self):
    """Saves current structural multi-column tree grids to database storage."""
    if not self.db_path:
        return
    try:
        self.tree_index.save_to_db(self.db_path)
        self.clear_session_backups()
        self._tree_modified = False
        QMessageBox.information(self, "Success", "Workspace changes successfully committed to database.")
    except Exception as e:
```

```
            QMessageBox.critical(self, "Database Error", f"Failed to save index database: {e}")

    def revert_session_changes(self):
        """Overwrites modified disk files back to pristine states and reloads editor text."""
        for original_path, backup_path in self.backup_registry.items():
            try:
                if os.path.exists(backup_path):
                    shutil.copy2(backup_path, original_path)
            except Exception as e:
                print(f"Failed to restore backup for {original_path}: {e}")

        self.clear_session_backups()

        if self.db_path and os.path.exists(self.db_path):
            self.tree_index.load_from_db(self.db_path)

        # Silently refresh open text canvas layers from disk directly
        for i in range(self.tabs.count()):
            tab_path = os.path.normpath(self.tabs.tabToolTip(i))
            editor = self.tabs.widget(i)

            # if os.path.exists(tab_path) and isinstance(editor, QPlainTextEdit):
            if os.path.exists(tab_path) and isinstance(editor, EditorTab):
                try:
                    with open(tab_path, 'r', encoding='utf-8') as f:
                        editor.blockSignals(True)
                        editor.setPlainText(f.read())
                        editor.blockSignals(False)
                except Exception as e:
                    print(f"Failed to refresh editor tab for {tab_path}: {e}")

        self._tree_modified = False
        QMessageBox.warning(self, "Reverted", "All files and index logs reverted to the previous commit.")

    def clear_session_backups(self):
        """Deletes orphan temporary files from storage to save disk space."""
        for backup_path in self.backup_registry.values():
            if os.path.exists(backup_path):
                try:
                    os.remove(backup_path)
                except Exception as e:
                    print(f"Failed to remove temporary file {backup_path}: {e}")

        self.backup_registry.clear()
        self.session_files.clear()

        if self.backup_dir and os.path.exists(self.backup_dir):
            try:
                if not os.listdir(self.backup_dir):
                    os.rmdir(self.backup_dir)
            except Exception as e:
                print(f"Failed to clean empty backup directory: {e}")

    # --- Find dialog for tabs ---
    def route_find_to_active_tab(self):
        active_tab = self.tabs.currentWidget()
        # Route action down to the unique local tab instance context
        if isinstance(active_tab, EditorTab):
            active_tab.toggle_find_dialog()

    # --- Application Lifecycle Overridden Events ---

    # Inside your MainWindow class (Chunk 6)
    def eventFilter(self, source, event):
        """Catch events and override wheel event to implement Ctrl + Mouse Wheel for font resizing."""
        if event.type() == QEvent.Wheel and event.modifiers() == Qt.ControlModifier:
            delta = event.angleDelta().y()

            if delta > 0:
                new_size = min(72, self.current_font_size + 1)
                self.update_font_size(new_size)
            elif delta < 0:
                new_size = max(6, self.current_font_size - 1)
                self.update_font_size(new_size)

            # Returning True consumes the event, stopping further propagation
            return True

        # Ensure this always executes and returns a boolean value
        return super().eventFilter(source, event)


if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = LatexEditor()
    window.show()
    sys.exit(app.exec())
```